

✓ Height & Weight Prediction — All Regression Models + Gradio GUI

Dataset: `Data_person.csv`

Target: **Weight**

Features: **Height** and **Sex**

```
# =====
# COMPLETE ONE-CELL COLAB CODE
# Height + Sex -> Weight using multiple regression models
# =====

!pip -q install gradio

import pandas as pd
import numpy as np
import pickle
import matplotlib.pyplot as plt
import gradio as gr

from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, r2_score

# -----
# 1. Upload dataset
# -----
print('Upload Data_person.csv')
uploaded = files.upload()

df = pd.read_csv('Data_person.csv')
print('\nDataset loaded successfully!')
print('Shape:', df.shape)
print('Columns:', df.columns.tolist())
display(df.head())

# -----
# 2. Basic information
# -----
print('\n==== DATASET INFORMATION ====')
print(df.info())
print('\nMissing values:')
print(df.isnull().sum())
print('\nDuplicate rows:', df.duplicated().sum())
print('\nStatistical summary:')
display(df.describe(include='all'))

# -----
# 3. Remove missing rows if any
# -----
df = df.dropna(subset=['weight', 'height', 'Sex']).copy()

# -----
# 4. Encode Sex
# -----
encoder = LabelEncoder()
```

```

df['Sex_Encoded'] = encoder.fit_transform(df['Sex'])

print('\n===== SEX ENCODING =====')
for label, value in zip(encoder.classes_, encoder.transform(encoder.classes_)):
    print(label, '->', value)

# -----
# 5. Visualization: Height vs Weight
# -----
plt.figure(figsize=(8, 5))
plt.scatter(df['height'], df['weight'], alpha=0.5)
plt.xlabel('Height')
plt.ylabel('Weight')
plt.title('Height vs Weight')
plt.grid(True, alpha=0.2)
plt.show()

# -----
# 6. Features and target
# -----
X = df[['height', 'Sex_Encoded']]
y = df['weight']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# -----
# 7. Scale features
# -----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -----
# 8. Regression models
# -----
models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'Lasso Regression': Lasso(alpha=0.1),
    'Decision Tree Regression': DecisionTreeRegressor(max_depth=6, random_state=42),
    'Random Forest Regression': RandomForestRegressor(n_estimators=200, random_state=42),
    'Gradient Boosting Regression': GradientBoostingRegressor(random_state=42),
    'SVR': SVR(kernel='rbf')
}

# -----
# 9. Train and evaluate all models
# -----
results = []
trained_models = {}

print('\n===== MODEL RESULTS =====')

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    pred = model.predict(X_test_scaled)
    trained_models[name] = model

    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2 = r2_score(y_test, pred)

    results.append({
        'Model': name,
        'RMSE': rmse,

```

```

    R2 Score : R2
    })

results_df = pd.DataFrame(results).sort_values('RMSE').reset_index(drop=True)
display(results_df)

# -----
# 10. Select best model
# -----
best_model_name = results_df.loc[0, 'Model']
best_model = trained_models[best_model_name]

print('\n=====')
print('BEST MODEL:', best_model_name)
print('RMSE:', round(results_df.loc[0, 'RMSE'], 4))
print('R2 Score:', round(results_df.loc[0, 'R2 Score'], 4))
print('=====')

# -----
# 11. Save model, scaler and encoder
# -----
with open('best_model.pkl', 'wb') as f:
    pickle.dump(best_model, f)

with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

with open('sex_encoder.pkl', 'wb') as f:
    pickle.dump(encoder, f)

print('\nSaved files:')
print('1. best_model.pkl')
print('2. scaler.pkl')
print('3. sex_encoder.pkl')

# -----
# 12. Prediction function
# -----
def predict_weight(sex, height):
    sex_encoded = encoder.transform([sex])[0]
    sample = np.array([[float(height), sex_encoded]])
    sample_scaled = scaler.transform(sample)
    prediction = best_model.predict(sample_scaled)[0]
    return round(float(prediction), 2)

# Test prediction
print('\nExample prediction:')
print(predict_weight(encoder.classes_[0], 170))

# -----
# 13. Gradio GUI
# -----
def gradio_predict(sex, height):
    if height is None:
        return 'Please enter height.'
    return f'{predict_weight(sex, height)} kg'

demo = gr.Interface(
    fn=gradio_predict,
    inputs=[
        gr.Dropdown(
            choices=encoder.classes_.tolist(),
            label='Sex'
        ),
        gr.Number(
            label='Height (cm)',

```

```
        minimum=50,  
        maximum=250,  
        value=170  
    )  
],  
outputs=gr.Textbox(label='Predicted Weight'),  
title='Height and Weight Prediction',  
description=f'Best model: {best_model_name}. Enter sex and height to predict weight.'  
)  
  
demo.launch(share=True)
```

